Program 1:

Question:

Demonstrate the steps for installation of R and R Studio. Perform the following:

a) **Assign different type of values to variables and display the type of variable. Assign different types such as Double, Integer, Logical, Complex and Character and understand the difference between each data type.**

b) **Demonstrate Arithmetic and Logical Operations with simple examples.**

c) **Demonstrate generation of sequences and creation of vectors.**

d) **Demonstrate Creation of Matrices**

e) **Demonstrate the Creation of Matrices from Vectors using Binding Function.**

f) **Demonstrate element extraction from vectors, matrices and arrays**

Solution:

Step 1: Download and Install R

1. Go to the CRAN (Comprehensive R Archive Network) website.
2. Choose a CRAN mirror location near you.
3. Download the R installer appropriate for your operating system (Windows, macOS, or Linux).
4. Run the installer and follow the installation instructions.

Step 2: Open R Console (RGui or RStudio)

- If you're using the default RGui on Windows, open it by searching for "R" in the Start menu. Alternatively, if you're using RStudio, open RStudio.

Step 3: Install Basic Packages

In the R console or RStudio, you can use the **install.packages()** command to install additional R packages

Let's install a few basic packages as examples:

```
# Install the dplyr package for data manipulation

install.packages("dplyr")

# Install the ggplot2 package for data visualization

install.packages("ggplot2")

# Install the tidyr package for data tidying

install.packages("tidyr")
```

Run these commands one by one, and R will download and install the specified packages

from the CRAN repository.

Step 4: Load Installed Packages (if needed)

After installation, you typically need to load the installed packages into your R session using the **library ()** function:

```
# Load the dplyr package
library(dplyr)
# Load the ggplot2 package
library(ggplot2)
# Load the tidyr package
library(tidyr)
```

1a) Assign different type of values to variables and display the type of variable. Assign different types such as Double, Integer, Logical, Complex and Character and understand the difference between each data type.

```
# Double
double_var <- 3.14
print(class(double_var))

# Integer
int_var <- 42L
print(class(int_var))

# Logical
logical_var <- TRUE
print(class(logical_var))

# Complex
complex_var <- 2 + 3i
```

```
print(class(complex_var))

# Character
char_var <- "Hello, World!" print(class(char_var))
```

1b. Demonstrate Arithmetic and Logical Operations with simple examples.

```
# Arithmetic operations
a <- 5
b <- 3
sum_ab <- a + b
diff_ab <- a - b
product_ab <- a * b
quotient_ab <- a / b

# Logical operations
x <- TRUE
y <- FALSE
and_result <- x && y # Logical AND
or_result <- x || y # Logical OR
not_x <- !x  # Logical NOT

print(paste("Sum: ", sum_ab))
print(paste("Difference: ", diff_ab))
print(paste("Product: ", product_ab))
print(paste("Quotient: ", quotient_ab))
print(paste("AND Result: ", and_result))
```

```
print(paste("OR Result: ", or_result))
print(paste("NOT x: ", not_x))
```

1c. Demonstrate generation of sequences and creation of vectors.

```
# Sequence from 1 to 10
seq_vector <- 1:10
# Sequence from 1 to 10 with a step of 2
seq_vector_step <- seq(1, 10, by = 2)
print(seq_vector)
print(seq_vector_step)
```

1d) Create a 3x3 matrix

```
mat <- matrix(1:9, nrow = 3, ncol = 3)
print(mat)
```

1e) Demonstrate the Creation of Matrices from Vectors using Binding Function.

Create two vectors

```
vec1 <- c(1, 2, 3)
vec2 <- c(4, 5, 6)
# Bind them as rows to create a matrix
mat_from_rows <- rbind(vec1, vec2)
# Bind them as columns to create a matrix
mat_from_cols <- cbind(vec1, vec2)
print(mat_from_rows)
print(mat_from_cols)
```

1f) Demonstrate element extraction from vectors, matrices and arrays

Vectors

```
vec <- c(10, 20, 30, 40, 50)
element <- vec[3] # Extract the third element

# Matrices
mat <- matrix(1:9, nrow = 3, ncol = 3)
element_mat <- mat[2, 2] # Extract element in the second row and second column
```

```
print(paste("Vector Element: ", element))
print(paste("Matrix Element: ", element_mat))
```

1. Vectors (1-Dimensional)

A vector is a flat sequence of elements. You extract elements by specifying a single integer index or a range inside square brackets []. Note that R uses 1-based indexing (the first element is at index

Create a sample vector

```
v <- c(10, 20, 30, 40, 50)
# Extract a single element (3rd position) v[3]
# Output: 30
# Extract a range of elements (2nd to 4th position)
v[2:4]
# Output: 20 30 40
# Extract specific non-contiguous elements
v[c(1, 5)]
# Output: 10 50
```

2. Matrices (2-Dimensional)

A matrix organizes elements into rows and columns. You extract elements using a comma-separated pair of indices: [row, column]. Leaving a field blank extracts the entire row or column.

Create a 3x3 matrix

```
m <- matrix(1:9, nrow = 3, ncol = 3)
# [,1] [,2] [,3]
```

[1,] 1 4 7

[2,] 2 5 8

[3,] 3 6 9

Extract a single element (Row 2, Column 3)

m[2, 3]

Output: 8

Extract an entire row (Row 2)

m[2,]

Output: 2 5 8

Extract an entire column (Column 1) m[, 1]

Output: 1 2 3

3. Arrays (multi-dimensional)

An array extends vectors and matrices into three or more dimensions. You extract elements by supplying indices for each dimension separated by commas: [dim1, dim2, dim3, ...]

Create a 3-dimensional array (2x2x2)

a <- array (1:8, dim = c (2, 2, 2))

Extract a single element (Row 1, Column 2, Layer 2)

a [1, 2, 2]

Output: 7

Extract a 2D slice from the first layer/matrix

A [, , 1]

Output:

[,1] [,2]

[1,] 1 3

[2,] 2 4

You extract elements from arrays in R using square brackets [] with comma-separated indices for each dimension.

Creating a Sample Array
First, create a 3D array with 2 rows, 3 columns, and 2 layers (matrices).

```
# Create data and define dimensions

data_vector <- 1:12

my_array <- array(data_vector, dim = c(2, 3, 2)) print(my_array)
```

1. Extracting a Single Element
To get one specific item, pass the exact index for each dimension: array_name[row, column, layer]

```
# Extract element at row 2, column 2, in the 1st layer

elem <- my_array[2, 2, 1]
 print(elem)

# Output: 3
```

3. Extracting Slices and Ranges
Use the colon : operator or index vectors (c()) to get ranges of elements.

```
# Extract rows 1 through 2, column 1 through 2, from the 1st layer

subset_data <- my_array[1:2, 1:2, 1]

print(subset_data)
```

4. Extracting Multiple Non-Contiguous Elements via an Index Matrix

```
# Define coordinates as a matrix (row, col, layer)


indices <- matrix(c(
        1, 1, 1, # 1st element: row 1, col 1, layer 1
```

```
 2, 3, 2 # 2nd element: row 2, col 3, layer 2),
ncol = 3, byrow = TRUE)

# Extract values using the index matrix

extracted_vals <- my_array[indices]
print(extracted_vals)
```

Program 2

Assess the Financial Statement of an Organization being supplied with 2 vectors of data: Monthly Revenue and monthly expenses for the financial year. You can create your own sample data vector for this experiment) Calculate the following financial metrics:

a. **Profit for each month.**
b. **Profit after tax for each month (Tax Rate is 30%).**
c. **Profit margin for each month equals to profit after tax divided by revenue.**
d. **Good Months – where the profit after tax was greater than the mean for the year.**
e. **Bad Months – where the profit after tax was less than the mean for the year.**
f. **The best month – where the profit after tax was max for the year.**
g. **The worst month – where the profit after tax was min for the year.**

Note:

a. **All Results need to be presented as vectors**
b. **Results for Dollar values need to be calculated with $0.01 precision, but need to be presented in Units of $1000 (i.e 1k) with no decimal points**
c. **Results for the profit margin ratio need to be presented in units of % with no decimal point.**

d. **It is okay for tax to be negative for any given month (deferred tax asset)**
e. **Generate CSV file for the data.**

```
# Monthly Revenue (in Dollars)

revenue <- c(145000, 132000, 158000, 142000, 170000, 165000,

        180000, 175000, 160000, 155000, 168000, 182000)
# Monthly Expenses (in Dollars)

expenses <- c(120000, 118000, 130000, 125000, 145000, 140000,

        150000, 148000, 138000, 140000, 142000, 150000)
# Tax Rate

tax_rate <- 0.30
```

2a. Profit for each month

```
profit <- revenue - expenses
```

2b. Profit After Tax

```
profit_after_tax <- profit - (profit * tax_rate)
```

2c. Profit Margin

```
profit_margin <- (profit_after_tax / revenue) * 100
```

2d. Good Months

```
mean_profit <- mean(profit_after_tax)
```

good_months <- profit_after_tax > mean_profit

2e. Bad Months

bad_months <- profit_after_tax < mean_profit

2f. Best Month

best_month <- profit_after_tax == max(profit_after_tax)

2g. Worst Month

worst_month <- profit_after_tax == min(profit_after_tax)

Convert Dollar Values into Thousands (k)

revenue_k <- round(revenue / 1000)

expenses_k <- round(expenses / 1000)

profit_k <- round(profit / 1000)

profit_after_tax_k <- round(profit_after_tax / 1000)

Profit Margin without decimal

profit_margin_percent <- round(profit_margin)

Display Results

cat("Revenue (k):\n") print(revenue_k)

cat("\nExpenses (k):\n") print(expenses_k)

cat("\nProfit (k):\n") print(profit_k)
cat("\nProfit After Tax (k):\n") print(profit_after_tax_k)

```
cat("\nProfit Margin (%):\n")

print(profit_margin_percent)
```

```
cat("\nGood Months:\n")

print(good_months)

cat("\nBad Months:\n")

print(bad_months)

 cat("\nBest Month:\n")

 print(best_month)

cat("\nWorst Month:\n") print(worst_month)

# Create Data Frame

financial_data <- data.frame( Revenue = revenue, Expenses = expenses,
Profit = profit,
Profit_After_Tax = round(profit_after_tax, 2),
Profit_Margin = round(profit_margin, 2),
Good_Month = good_months,
Bad_Month = bad_months,
Best_Month = best_month,
Worst_Month = worst_month
)

# Display Data Frame

print(financial_data)

  # Export Data to CSV

write.csv(financial_data, file = "Financial_Statement.csv", row.names =
FALSE)
```

```
cat("\nCSV file 'Financial_Statement.csv' has been created
successfully.\n")
```